

PRODUCT & ARCHITECTURE GUIDE

AI Tutor

Learning platform foundation, product flows and Architecture V1.0

A practical guide to what the product is building, how a learner moves through it, and what is implemented today.

Source of truth: numbered Markdown files in /docs

01. Product & Principles

Product promise

AI Tutor giúp người học nói tiếng Anh thường xuyên, nhận một phản hồi hữu ích ngay sau khi nói, thử lại khi còn trong ngữ cảnh và tự động có kế hoạch ôn vào lúc phù hợp. Sản phẩm không dừng ở chat AI; mỗi lượt thực hành phải đóng góp vào tiến độ học.

Người dùng chính

- Người đi làm Việt Nam muốn nói tự tin trong tình huống hàng ngày hoặc công việc.
- Người cần một lộ trình rõ ràng cho IELTS/TOEIC sau khi core learning loop ổn định.
- Người học bận, cần bắt đầu bài 3 phút bằng một thao tác.

Learning loop

Chọn hoặc tiếp tục hoạt động

- nghe / đọc / nói / viết
- assessment
- một feedback trọng tâm
- retry ngay nếu hữu ích
- Knowledge Delta
- recommendation và lịch ôn

Principles

1. **Voice-first, not voice-only.** Người dùng có thể nói hoặc gõ; learning record giống nhau.
2. **One meaningful correction.** Không làm người học ngợp bằng danh sách lỗi.
3. **Retry while context is fresh.** Câu sửa phải dẫn đến hành động nói lại.
4. **Personal progress, not generic streaks.** Streak chỉ là presentation; mastery là dữ liệu học tập.
5. **Fast by default.** Tutor, voice và content không được làm người dùng chờ vô ích.
6. **Explainable decisions.** Feedback, recommendation và score đều phải có reason codes và version.

Không phải mục tiêu hiện tại

- Không xây một agent tự do có thể dạy mọi thứ mà không kiểm soát.
- Không làm nhiều mode IELTS/TOEIC trước khi loop speaking cơ bản được chứng minh.
- Không chia microservice chỉ để có sơ đồ đẹp.

02. User Experience & Flows

First-time experience

Mở app

- chọn mục tiêu ngắn gọn

- chọn hoặc nhận đề xuất một topic
- tutor bắt đầu
- người dùng nói một câu
- nhận insight và lựa chọn retry / continue

Mục tiêu: người dùng nói câu đầu tiên trong dưới hai phút. Setting, chọn voice và tính năng nâng cao chỉ mở khi cần.

Daily home

Nút chính luôn là **Continue learning**. Nó dẫn đến activity tốt nhất dựa trên review đến hạn, knowledge yếu, mục tiêu, lịch sử gần đây và thời gian sẵn có.

Live speaking

1. Tutor nêu ngữ cảnh và câu hỏi.
2. Người học nói hoặc gõ.
3. App hiển thị transcript và tutor trả lời streaming.
4. Learning insight hiện độc lập với tutor reply: score ngắn, một feedback, nút **Try again**.
5. Người dùng retry hoặc tiếp tục cuộc hội thoại.

Finish state

Khi kết thúc, người dùng chỉ cần thấy: điều đã làm tốt, một điểm cần ôn, từ/cấu trúc được lưu và bước học hợp lý tiếp theo. Không dùng popup dày đặc hoặc dashboard nặng làm gián đoạn.

Presentation Engine boundary

Presentation Engine quyết định **cách hiển thị** state: animation, overlay, accessibility, responsive layout và transition. Nó không tính mastery, score hay recommendation.

Accessibility & performance

- Luôn có text fallback cho audio và microphone.
- UI đáp ứng trước; audio/TTS bắt đầu sau khi text đầu tiên được paint.
- 3D tutor là progressive enhancement, không chặn bài học.
- Hiệu ứng/reward tôn trọng reduced motion và không can thiệp tác chính.

03. Capabilities & Content Engine

Capability trước, product pack sau

Core không biết IELTS, Kids hay Business là gì. Core chỉ biết capability và contract của capability.

Capability	Input	Assessment	Output học tập
Speaking	voice/text	rule, AI hoặc human	fluency, grammar, pronunciation deltas
Reading	text	quiz/rule	comprehension and vocabulary deltas

Capability	Input	Assessment	Output học tập
Listening	audio	alignment/quiz	listening deltas
Writing	text	rubric/AI	structure and grammar deltas
ImageQuiz	image + choice	auto	vocabulary/concept deltas

Content Engine

ContentBlock
→ versioned paragraph / audio / image / prompt material
→ Activity
→ KnowledgeNode links
→ render through a Capability

Reading, listening, roleplay, flashcard và quiz là các cách render activity khác nhau. Nội dung phải versioned để người học giữa một plan không bị đổi bài bất ngờ.

Product packs

Pack	Đăng ký
IELTS	speaking assessment, band rubric, cue card, essay, listening
TOEIC	timed quiz, listening, reading, score rubric
Business	roleplay, email, meeting, presentation
Kids	safe speaking, listening, image quiz, high-reward presentation policy

Plugin đăng ký capability, content taxonomy, rubric và policy - không đăng ký màn hình riêng hoặc sửa core.

CMS direction

CMS đến sau core loop. Editor tạo block, gắn knowledge, preview activity, version và publish. AI có thể hỗ trợ draft content nhưng con người/policy chịu trách nhiệm publish.

04. Learning & Knowledge Engine

Knowledge-centric, not lesson-centric

Lesson chỉ là một cách đưa knowledge đến người học. Một node như spill, past simple hoặc car vocabulary có thể xuất hiện trong speaking, reading, listening và quiz.

KnowledgeNode
→ KnowledgeNodeVersion
→ UserKnowledge
→ mastery / confidence / exposure / error / nextReviewAt

UserKnowledge

UserKnowledge là state hiện tại của một learner với một node. Nó gồm mastery, confidence, exposure count, error count và thời gian review tiếp theo. Knowledge Engine là nơi duy nhất cập nhật state này.

Learning Engine

Learning Engine nhận AssessmentResult, tạo KnowledgeDelta, quản lý attempt/session/SRS và learning plan. Nó không tự chấm bài và không tự tính mastery.

```
Attempt
→ AssessmentResult
→ KnowledgeDelta
→ Knowledge Engine applies delta
→ updated snapshot
→ Recommendation
```

Recommendation

Online decision

Chọn next action rất nhanh: review due, weak node, goal match và repetition penalty. Kết quả là một Decision, không chỉ là activity ID.

Offline plan

Sau này worker dùng insight, interest, schedule và retention pattern để chuẩn bị daily/weekly plan trước khi người dùng mở app.

SRS

SRS là kết quả của knowledge state và learning plan, không phải một flashcard module độc lập. V1 đã lưu nextReviewAt; UI review đầy đủ là phase tiếp theo.

05. AI & Assessment Engine

AI is a provider, not the core

AI không sở hữu learning state. AI Engine nhận workflow, prompt version, policy và schema; sau đó trả dữ liệu hợp lệ cho Assessment Engine hoặc tutor presentation.

```
Workflow
→ prompt + model router + policy
→ JSON schema validator
→ AssessmentResult or tutor response
```

Assessment Engine

Mọi hình thức chấm phải chuẩn hóa thành AssessmentResult:

- speaking: rule baseline, LLM, pronunciation model hoặc human review;
- quiz: auto score;

- reading: rule/quiz;
- listening: speech alignment/quiz;
- writing: rubric + LLM.

Learning Engine chỉ tiêu thụ kết quả chuẩn hóa, không quan tâm provider nào tạo ra nó.

Structured output

Không parse tag kiểu <feedback>. Assessment/tutor workflows phải trả JSON có schema, feedback ưu tiên, retry prompt, reason codes và version references.

Current baseline

Speaking Slice đang dùng rule-based assessment để không cộng thêm LLM latency vào lượt nói. Nó kiểm tra độ dài, connected idea và lỗi quá khứ cơ bản. Đây là baseline có chủ đích; AI assessment được thêm sau khi replay/evaluation workflow sẵn sàng.

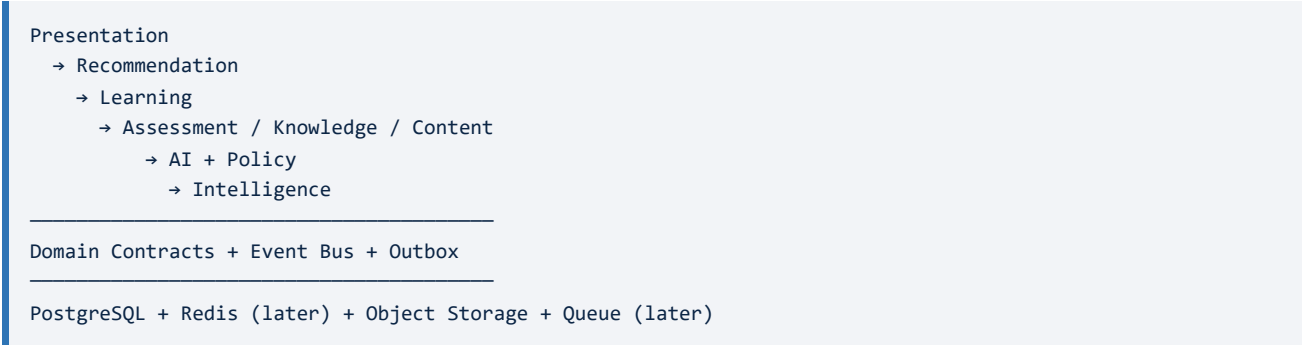
Guardrails

Policy Engine cung cấp kid-safe topics, level/difficulty, rubric policy, experiment flags và subscription capability. Prompt, recommendation và presentation cùng đọc policy snapshot thay vì tự suy đoán.

06. Architecture V1.0

Decision

Architecture V1.0 là modular monolith với engine boundaries rõ ràng. Các engine là ranh giới ownership, chưa phải microservice.



Engine responsibilities

Engine	Owns
Content	blocks, activities, versions, publish
Knowledge	graph, snapshots, mastery, confidence, decay
Learning	sessions, attempts, deltas, SRS, plans
Assessment	normalized assessment results
AI	workflow registry, model routing, validation, eval

Engine	Owns
Recommendation	next activity and longer-term plan decisions
Presentation	UI runtime and accessible delivery
Policy	segment, safety, feature and experiment rules
Intelligence	insight, cohort, retention and experiment evaluation

Non-negotiable laws

1. Business logic exists in one owner only.
2. An engine does not read/write another engine's database implementation.
3. Cross-boundary communication uses immutable contracts, commands/queries or past-tense events.
4. Event Bus announces facts; it does not hide synchronous business logic.
5. Application layer may orchestrate a vertical slice but owns no domain rule.

Why modular monolith first

It keeps deployment, debugging and iteration fast while dependency boundaries are enforced in CI. Extraction happens only when independent load, ownership or release cadence makes it necessary.

07. Contracts, Data & Events

Core contracts

Contract	Owner	Purpose
Capability	Content	defines activity input/output contract
AssessmentResult	Assessment	immutable normalized evaluation
KnowledgeDelta	Learning	requested change after an attempt
KnowledgeSnapshot	Knowledge	current learner state for a node
RecommendationDecision	Recommendation	explainable next action
LearningPlan	Learning	daily/weekly activity plan
PolicyDecision	Policy	resolved rules for current context
LearningInsight	Intelligence	aggregated meaningful observation

LearningContext

Every command has a small immutable context: actor/learner, locale, timezone, capability, policy snapshot, experiment flags, subscription and optional session reference. It is not a place for database clients, mutable state or secrets.

VersionRef

Content, knowledge, capability, rubric, prompt and policy use the same audit shape:

```
VersionRef { id, type, version, createdAt }
```

Assessments pin the relevant refs. A replay creates a new assessment; it does not overwrite history.

Events and outbox

TurnCompleted, KnowledgeUpdated and RecommendationCreated are written with the domain transaction to OutboxEvent. A future worker publishes unpublished events to a real broker. Consumers react independently.

Idempotency

Every turn has a unique idempotency key. Retrying the same key must not create duplicate turn, assessment, delta, decision or outbox events.

Audit and explainability

Decision includes outcome, reason codes, confidence, input references, policy/experiment references and versions. This makes recommendation and assessment debuggable without storing unnecessary duplicate PII.

08. Speaking Slice: Car - Current State

Purpose

This is the first real vertical slice that proves the learning platform contract, using the existing Car topic.

Implemented

```
Topic Car
→ LearningSession
→ text or voice transcript
→ LearningTurn
→ AssessmentResult
→ KnowledgeDelta
→ UserKnowledge update
→ RecommendationDecision
→ OutboxEvent
→ learning-update UI feedback
```

Database

The PostgreSQL migration 20260719074840_architecture_v1_speaking_slice is applied. The schema includes sessions, turns, immutable assessments, knowledge nodes/versions, user knowledge, activities, decisions and outbox events.

UI

The user sees a short Practice Insight after a turn: score, one focused improvement and a Try again action. It is deliberately separate from the streamed tutor answer to preserve responsiveness.

Assessment baseline

The baseline is rule-based: response length, connected ideas and a simple past-tense signal. This creates normalized data without another LLM request.

Safety

If learning persistence is unavailable during a deployment, the legacy realtime tutor still works. Learning failure is logged and does not fail the conversation.

Verified

- Prisma migration status is up to date.
- Database smoke test creates and verifies session, assessment, knowledge, recommendation and three outbox events.
- Architecture dependency check passes.
- Shared contracts, frontend type check and production build pass.

Not yet complete

- Reconnect/resume of an active learning session.
- Replay command for an assessment with a newer rubric/prompt.
- Outbox publisher worker and distributed queue.
- Full SRS review UI, auth and learner profile.
- AI-based assessment workflow/evaluation suite.

09. Delivery Roadmap

Rule

Do not add many content modes before the current vertical slice meets its Definition of Done.

Phase A - Complete Speaking Slice

1. Persist and resume session after reconnect.
2. Add assessment replay with superseding history.
3. Add minimal outbox publisher, structured logs and trace/correlation IDs.
4. E2E test: Car, wrong past tense, retry, updated state and recommendation.

Phase B - Learning habit

1. User authentication/profile and goal setup.
2. Review queue and SRS UI.
3. Daily plan and continue-learning home.

4. Vocabulary notebook linked to KnowledgeNode.

Phase C - Capability proof

1. Reading slice: content + quiz assessment.
2. Listening slice: audio + alignment/quiz assessment.
3. Writing slice: rubric-based AI assessment.

No new core contract unless one of these slices proves a gap.

Phase D - Production platform

1. Redis for distributed session/cache/rate limit.
2. Queue/workers for speech, AI, outbox and plans.
3. Object storage for audio.
4. OpenTelemetry, metrics, dashboards and alerting.

Phase E - Growth

CMS, content packs, experiments, subscription/billing, referral and deeper recommendation follow only after the learning loop has reliable data.

10. Operations & Quality

Performance contract

- Initial tutor visual/voice preparation must not block first useful screen.
- Text appears before first TTS playback.
- Chat render is throttled; 3D is lazy-loaded and bounded by device capability.
- Content, dictionary and static assets use appropriate cache boundaries.

Reliability contract

- Idempotency for retryable turns.
- Transactional outbox for facts needing asynchronous delivery.
- No loss of active session after reconnect is the next acceptance target.
- Provider failure must degrade gracefully; realtime tutor is not brought down by learning persistence failure.

Architecture tests

`npm run architecture:check --prefix backend` blocks engine-to-engine implementation imports. It is a starting guardrail, expanded as engines grow.

Required observability

Every production turn should be traceable by request/session/learner correlation IDs. Minimum signals: STT latency, first LLM token, TTS latency, turn completion, assessment result, event publishing failure

and client errors.

Scale path

Start with one modular backend and PostgreSQL. Add Redis/session coordination, queue workers and object storage based on observed load. Do not introduce graph DB, microservices or ML recommendation without an RFC backed by a measured limitation.

Definition of Done for a capability slice

- Functional end-to-end user flow.
- Immutable assessment and version references.
- Knowledge update and explainable recommendation.
- Idempotency, audit/outbox and failure behavior.
- Architecture dependency checks.
- Unit/smoke/E2E coverage proportional to risk.

11. Glossary

Term	Meaning
Activity	A runnable learning unit rendered through a capability.
AssessmentResult	Immutable standardized result of evaluating an attempt.
Attempt / LearningTurn	A learner's submitted voice or text response.
Capability	Reusable contract such as Speaking, Reading or Writing.
ContentBlock	Versioned source content: text, audio, image or prompt material.
Decision	Explainable result with outcome, reason codes, confidence and version refs.
Engine	A business ownership boundary, not automatically a service.
Knowledge Delta	A requested change to learner knowledge after an assessment.
Knowledge Node	A unit of knowledge: word, grammar, skill, concept or topic.
LearningContext	Immutable request context with learner, locale, policy and capability refs.
Outbox Event	A durable event written with the transaction, published later by a worker.
Policy	Rules by learner segment/capability: safety, difficulty, feature flags and entitlement.
Recommendation	A decision about the best next learning action.
Replay	Re-assessing a past attempt with a newer workflow without changing old history.
SRS	Spaced repetition scheduling based on learner knowledge state.
VersionRef	Common identifier for a versioned content/rubric/prompt/policy/capability artifact.